大家好，我是 Guide。赶在年前，终于完成了本项目的 Redis 常见面试题的系统整理。

这篇内容覆盖了 **Redis 基础原理、异步消息队列（Redis Stream）、分布式限流（AOP + Lua + 滑动窗口）、分布式缓存与会话管理、Redisson 实战应用** 等核心模块。更重要的是，每道题都配有**本项目中的实际应用案例**和**踩坑经验**，让你不仅"知道是什么"，更清楚"怎么用好"。

由于星球的 Markdown 显示目前做的还一般，如果想要更好的体验，建议前往阅读语雀：https://t.zsxq.com/dQNVc ，放在《SpringAI 智能面试平台+RAG 知识库》实战项目教程的「面试篇」。

Redis 基础

什么是 Redis?

Redis（**RE**mote **DI**ctionary **S**erver）是一个基于 C 语言开发的开源内存数据库，支持持久化。与传统数据库不同的是，Redis 的数据主要保存在内存中，因此读写速度非常快，被广泛应用于分布式缓存方向。

Redis 内置了多种数据类型实现（比如 String、Hash、List、Set、Sorted Set、Stream、Bitmap、HyperLogLog、GEO）。并且，Redis 还支持事务、持久化、Lua 脚本、发布订阅模型、多种集群方案（Redis Sentinel、Redis Cluster）。

☆Redis 为什么这么快?

Redis 内部做了非常多的性能优化，比较重要的有下面 4 点：

1. **纯内存操作**：这是最主要的原因。Redis 数据读写操作都发生在内存中，访问速度是纳秒级别，而传统数据库频繁读写磁盘的速度是毫秒级别，两者相差数个数量级。
2. **高效的 I/O 模型**：Redis 使用单线程事件循环配合 **I/O 多路复用**技术，让单个线程可以同时处理多个网络连接上的 I/O 事件，避免了多线程模型中的上下文切换和锁竞争问题。虽然是单线程，但结合内存操作的高效性和 I/O 多路复用，使得 Redis 能轻松处理大量并发请求。
3. **优化的内部数据结构**：Redis 提供多种数据类型，其内部实现采用高度优化的编码方式（如 ziplist、quicklist、skiplist、hashtable 等）。Redis 会根据数据大小和类型动态选择最合适的内部编码。
4. **简洁高效的通信协议**：Redis 使用的是自己设计的 RESP (REdis Serialization Protocol) 协议。这个协议实现简单、解析性能好，并且是二进制安全的。

☆为什么用 Redis 而不用本地缓存?

特性	本地缓存	Redis
数据一致性	多服务器部署时存在数据不一致问题	数据一致
内存限制	受限于单台服务器内存	独立部署，内存空间更大
数据丢失风险	服务器宕机数据丢失	可持久化，数据不易丢失
管理维护	分散，管理不便	集中管理，提供丰富的管理工具
功能丰富性	功能有限，通常只提供简单的键值对存储	功能丰富，支持多种数据结构和功能

本项目中的实际应用：

- **面试会话缓存**：使用 Redis 存储面试会话状态，替代了原有的 `ConcurrentHashMap`，支持多实例部署，确保会话数据在不同服务实例间共享。

```
// InterviewSessionCache.java
public class InterviewSessionCache {
    private static final String SESSION_KEY_PREFIX = "interview:session:";
    private static final Duration SESSION_TTL = Duration.ofHours(24);

    public void saveSession(String sessionId, CachedSession session) {
        String key = SESSION_KEY_PREFIX + sessionId;
        redisService.set(key, toJson(session), SESSION_TTL);
    }
}
```

☆Redis Stream 异步任务队列（本项目核心）

你的项目哪里用到了 Redis Stream?

项目中有三个典型的异步任务场景：

场景	说明	耗时
知识库向量化	将上传的文档切分、生成向量嵌入并存储到 pgvector	5-30 秒
简历 AI 分析	调用 LLM 对简历进行评分和建议生成	5-15 秒
面试报告生成	对面试会话进行综合评估，生成详细报告	5-20 秒

这些操作都涉及外部 API 调用（向量化模型、LLM），响应时间不稳定。如果采用同步处理，用户上传文件后需要长时间等待，体验极差，且容易触发 HTTP 超时。

为什么选择 Redis Stream?

在做架构设计时，千万不要为了炫技而引入复杂性。我们需要在**性能、运维成本和业务规模**之间寻找平衡。

特性	Redis Stream	Redis List	Redis pub/sub	Kafka	RabbitMQ
消费者组	✅ 原生支持	❌ 需自己实现	❌ 不支持 (广播模式)	✅ 支持	✅ 支持
消息确认	✅ ACK 机制	❌ 无 (需业务层处理)	❌ 无	✅ 支持	✅ 支持
消息持久化	✅ 支持	✅ 支持	❌ 不支持 (掉线即丢失)	✅ 支持	✅ 支持
消息回溯	✅ 支持 (基于 ID)	❌ 不支持 (出队即删)	❌ 不支持	✅ 支持	❌ 不支持
部署复杂度	低 (复用现有 Redis)	低	低	高	中
运维成本	低	低	低	高	中
适用规模	中小规模	简单队列	实时通知/即时通信	大规模	中大规模

Redis Stream 是 Redis 5.0 引入的数据结构，专为消息队列场景设计。对于我们的项目而言：

1. **复用现有基础设施**：项目已经使用 Redis 做缓存，无需额外部署消息队列。
2. **消费者组支持**：天然支持多实例部署，消息只会被一个消费者处理。
3. **消息确认机制**：通过 ACK 机制确保消息不丢失。
4. **轻量级**：相比 Kafka/RabbitMQ，运维成本更低。

我在 [Redis 常见面试题总结(上)](#)这篇文章中详细提到过如何基于 Redis 实现消息队列，还对比了 Redis List 和发布订阅 (pub/sub) 实现消息队列的差别。

为什么不用 PostgreSQL 做队列?

PostgreSQL 从 9.5 版本开始，可以通过 `SELECT ... FOR UPDATE SKIP LOCKED` 实现事务性队列。我们评估过这个方案，但最终选择 Redis Stream，主要基于**压力隔离**考量：

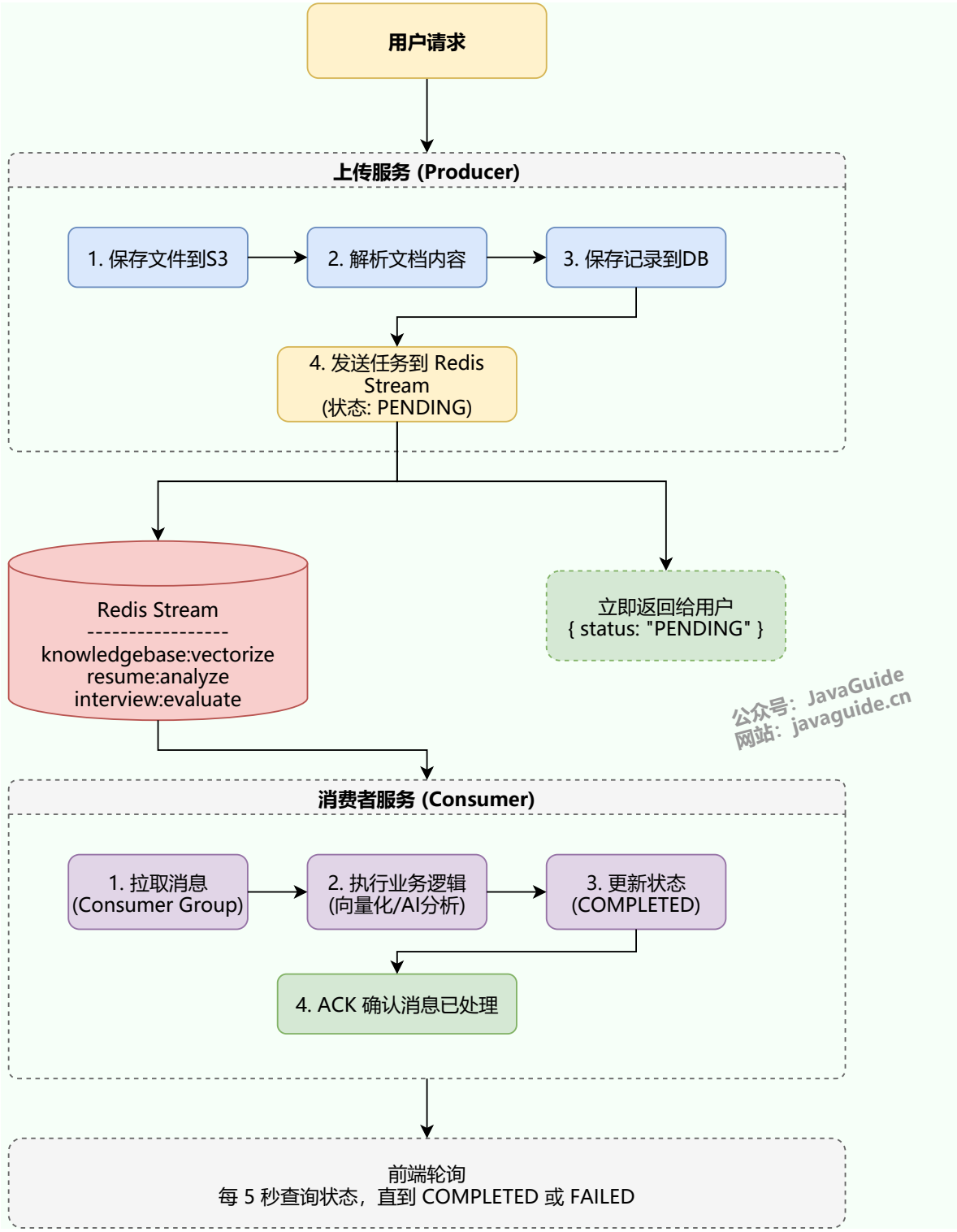
- 向量化任务是**高并发、写密集**的过程，大量的入队、出队、删除操作会产生海量 WAL 日志写入和表膨胀
- 在我们的设计中，PostgreSQL 承载的是用户前端的**核心查询负载**，不希望后台任务干扰用户体验
- 引入 Redis Stream 相当于做了一层**物理隔离**

那 Redis 岂不是多了个高可用风险点?

是的，引入中间件确实增加了链路复杂度。但考虑到 Redis 已经是系统中的标准组件（用于缓存），且本身支持主从复制或哨兵模式（Sentinel），这个成本相对于它带给主数据库的保护来说是值得的。

Redis 异步处理的整体流程是怎样的?

一句话描述：用户请求 → 生产者写入 Redis Stream → 消费者异步处理 → 更新状态 → 前端轮询获取结果。

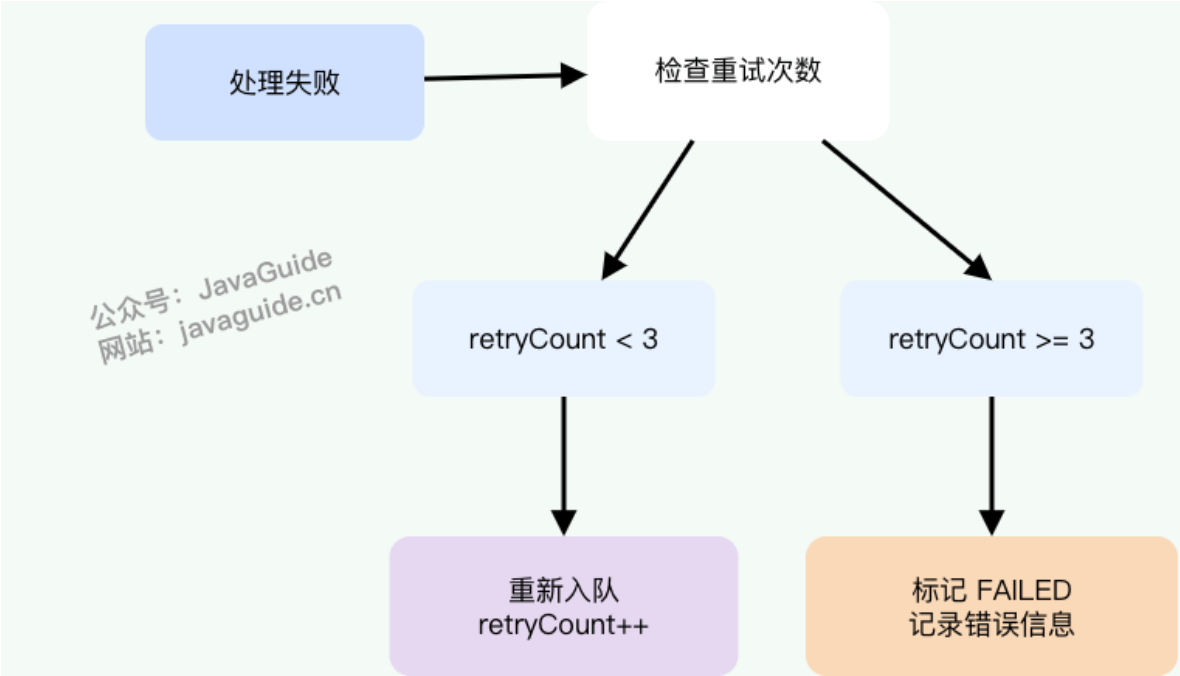


如果任务处理失败怎么办？

本项目实现了自动重试和手动重试功能。

自动重试

当任务处理失败时，如果未超过最大重试次数（默认 3 次），消费者会将任务重新发送到 Stream：



为了避免瞬时高峰导致雪崩，可扩展为指数退避（如 1s / 5s / 30s）。如果超过最大重试次数，更新数据库状态为 FAILED，不再重试。

手动重试

对于已标记为 FAILED 的任务，提供手动重试 API：

```
// Controller
@PostMapping("/api/knowledgebase/{id}/revectorize")
public Result<Void> revectorize(@PathVariable Long id) {
    uploadService.revectorize(id);
    return Result.success(null);
}

// Service
@Transactional
public void revectorize(Long kbId) {
    KnowledgeBaseEntity kb = knowledgeBaseRepository.findById(kbId)
        .orElseThrow(() -> new BusinessException(ErrorCode.NOT_FOUND, "知识库不存
在"));

    // 重新下载并解析文件
    String content = parseService.downloadAndParseContent(
        kb.getStorageKey(), kb.getOriginalFilename());

    // 重置状态
    kb.setVectorStatus(VectorStatus.PENDING);
    kb.setVectorError(null);
    knowledgeBaseRepository.save(kb);

    // 发送新任务
    vectorizeStreamProducer.sendVectorizeTask(kbId, content);
}
```

Redis Stream 使用有没有遇到什么坑？

坑 1：Stream 无限增长导致内存耗尽

现象：

Redis 中某个 Stream 消息无限增长，最终 Redis 内存耗尽。

原因：

一个常见的误区：`XACK` 只是确认消息已被消费，不会删除 Stream 里的消息条目。

- `XADD`：写入消息到 Stream
- `XREADGROUP`：消费者组读取消息
- `XACK`：确认消费（把消息从消费者组的 **PEL** / Pending Entries List “待处理列表”里移除）
- `XDEL`：从 Stream 中删除指定消息条目
- `XTRIM` / `MAXLEN`：裁剪 Stream（限制 Stream 长度，删除较旧的条目）

如果你既没有 `XDEL`，也没有 `XTRIM/MAXLEN`，那么 **Stream 里的历史消息会持续累积**，占用内存/磁盘。生产环境中，最推荐的方式是在写入时直接指定 **MAXLEN**，实现类似于定长环形队列的效果。

> 另外还有一种“堆积”是 **PEL 堆积**：消费者没有 `XACK`，导致待确认（pending）的消息越来越多。两者要区分排查。

解决方案：

发送消息时添加 **MAXLEN 限制**，自动裁剪旧消息：

```
// 修复前
stream.add(StreamAddArgs.entries(message));

// 修复后（自动裁剪超过 1000 条的旧消息）
stream.add(StreamAddArgs.entries(message)
    .trimNonStrict().maxLen(1000));
```

- `trimNonStrict()`：使用近似裁剪（`~`），性能更好
- `maxLen(1000)`：保留最新 1000 条消息

坑 2：删除实体后异步任务报错

问题：

后台日志频繁出现 `简历不存在: ID=35` 的 Error。检查发现，这是由于用户删除了简历，但分析任务还在跑。

原因：

这是导致数据不一致的典型问题：

1. 用户上传简历 → 发送分析任务到 Redis Stream
2. 分析失败 → 消息进入 pending/等待重试
3. 用户删除简历 → 数据库记录已删除
4. 消费者重试处理 → 找不到简历 → 报错

解决方案：

把“生命周期校验”放在异步任务处理的最前面，并区分：

- **不可恢复错误**（实体不存在、参数非法）→ 记录后 ACK/丢弃
- **可恢复错误**（临时网络故障、依赖服务超时）→ 不 ACK，让其重试或进入重试队列

示例（用一次查询代替 `existsById + findById` 两次查询）：

```java
private void processMessage(StreamMessageId messageId, Map<String, String> data)
{
    Long resumeId = Long.parseLong(data.get("resumeId"));

    var resumeOpt = resumeRepository.findById(resumeId);
    if (resumeOpt.isEmpty()) {
        // 不可恢复：实体已被用户删除（或数据已不存在）
        log.warn("检测到实体已被删除，跳过异步任务: resumeId={}", resumeId);
        ackMessage(messageId); // 必须 ACK，否则会反复重试造成噪音与堆积
        return;
    }

    try {
        Resume resume = resumeOpt.get();
        // 继续业务逻辑...
        ackMessage(messageId);
    } catch (TransientDependencyException e) {
        // 可恢复错误：不 ACK，让其重试（或转入重试/死信机制）
        log.warn("依赖异常，等待重试: resumeId={}, msgId={}", resumeId, messageId,
e);
        throw e;
    } catch (Exception e) {
        // 根据你的策略决定是否 ACK/重试/转死信
        log.error("处理失败: resumeId={}, msgId={}", resumeId, messageId, e);
        throw e;
    }
}
```

坑 3：忘记 ACK 导致消息重复消费

问题：处理失败时只做了重试入队，忘记 ACK 原消息，导致原消息在 Pending List 中无限堆积。

解决方案：无论成功失败都要 ACK 原消息：

```java
try {
    // 业务逻辑...
    ackMessage(messageId);
} catch (Exception e) {
    if (retryCount < MAX_RETRY_COUNT) {
        retryMessage(...);  // 重新入队
    } else {
        updateStatus(FAILED);
    }
    ackMessage(messageId);  // 🔑 关键：失败也要 ACK
}
```

Redis Stream 是 at-least-once 模型，如果未 ACK，消息会一直存在于 PEL 中，可能被重复投递。因此必须在“最终决策点”统一 ACK，而不是在多处散落 ACK。

☆分布式限流（本项目核心）

什么是服务限流？为什么需要限流？

服务限流是指对系统的请求速率进行控制，防止瞬时大量请求击垮系统的一种保护机制。

为什么需要限流：

1. **保护系统稳定性**：软件系统的处理能力是有限的，超过其承载能力会导致系统崩溃或响应缓慢。
2. **防止资源耗尽**：避免数据库连接池、线程池等关键资源被耗尽。
3. **保证服务质量**：通过牺牲部分请求来保证大部分用户的正常访问。
4. **防御恶意攻击**：防止恶意用户通过大量请求进行 DDoS 攻击。

现实类比： 就像景区限流一样，虽然会让部分游客无法立即进入，但能保证景区内游客的游览质量和安全。

常见限流算法有哪些？

- **固定窗口**：将时间划分为固定大小的窗口，在每个窗口内限制请求的数量或速率，即固定窗口计数器算法规定了系统单位时间处理的请求数量。
- **滑动窗口**：比于固定窗口计数器算法的优化在于：**它把时间以一定比例分片** 。
- **漏桶**：往桶中以任意速率流入水，以一定速率流出水。当水超过桶流量则丢弃，因为桶容量是不变的，保证了整体的速率。
- **令牌桶**：和漏桶算法算法一样，我们的主角还是桶（这限流算法和桶过不去啊）。不过现在桶里装的是令牌了，请求在被处理之前需要拿到一个令牌，请求处理完毕之后将这个令牌丢弃（删除）。我们根据限流大小，按照一定的速率往桶里添加令牌。如果桶装满了，就不能继续往里面继续添加令牌了。

关于常见的限流算法（如令牌桶、漏桶）和限流技术方案的详细介绍，可以参考JavaGuide 的《服务限流详解》。

何时选择分布式限流？

在讨论具体实现之前，我们需要明确分布式限流的适用场景。

以本项目为例，如果它始终是**单体应用**且只部署**单个实例**，那么引入基于 Redis 的分布式限流可能并非最优解。在这种单实例场景下，使用进程内的限流库，如 Google Guava 的 RateLimiter、Bucket4j 或 Resilience4j，通常是更轻量、高效和节省成本的选择。

然而，当应用需要**水平扩展**（即部署多个实例以承载更高流量）时，分布式限流就变得至关重要。想象一下，如果限制某个用户每秒只能访问 5 次，但在 3 个实例上各自使用内存限流器，用户实际可能达到 15 次/秒的访问速率，远超预期。**利用 Redis 作为共享的、集中式的状态存储，通过 Lua 脚本原子操作确保所有应用实例都遵循统一的限流规则，从而实现精确的全局速率控制。**

> **面试提示：** 如果面试官问及为何在单体项目中考虑使用分布式限流，务必能够清晰阐述是为未来的水平扩展做准备，或明确指出当前场景下更适合单机限流方案。理解方案的适用边界，避免留下技术选型不当的印象。

单体限流和分布式限流的区别是？

对比维度	单机限流（如 Guava RateLimiter）	分布式限流（如 Redis + Lua）
实现原理	进程内内存维护计数器	Redis 作为共享存储
适用场景	单实例应用	多实例集群部署
性能开销	极低（内存操作）	中等（网络 I/O）
数据一致性	实例间独立，无法协同	全局统一限流
运维成本	无需额外组件	需要 Redis 服务
扩展性	无法水平扩展	支持水平扩展
典型工具	Guava、Bucket4j、Resilience4j	Redis + Lua、Sentinel、Kong 网关

为什么要用 AOP + 注解实现限流?

限流属于横切关注点，使用 AOP 可以解耦业务逻辑，提高可维护性和扩展性。

限流本质是保障系统稳定性，而不是业务功能的一部分。

如果把限流代码直接写在 Controller/Service 里：

- 会污染业务代码
- 重复逻辑多
- 难以统一修改

AOP 实现关注点分离：

- 在方法执行前统一拦截
- 在方法执行后统一处理结果
- 不侵入业务代码

当前的流程是：

```
@RateLimit 注解 -> AOP 切面拦截 -> 执行 Lua 限流 -> 决定放行或降级
```

这种使用比手动调用限流 API 的方式更优雅直观一些，可维护性也更高。

使用 AOP 前（手动调用）：

```java
@PostMapping("/api/resumes/upload")
public Result<Map<String, Object>> uploadAndAnalyze(@RequestParam("file")
MultipartFile file) {
    // 手动调用限流逻辑（业务侵入）
    if (!rateLimiter.tryAcquire("upload", 5, 1, TimeUnit.SECONDS)) {
        throw new RateLimitExceededException("请求过于频繁");
    }

    // 业务逻辑
    Map<String, Object> result = uploadService.uploadAndAnalyze(file);
    return Result.success(result);
}
```

使用 AOP 后（声明式）：

```java
@PostMapping("/api/resumes/upload")
@RateLimit(dimensions = {Dimension.GLOBAL, Dimension.IP}, count = 5)
public Result<Map<String, Object>> uploadAndAnalyze(@RequestParam("file")
MultipartFile file) {
    // 纯粹的业务逻辑，限流逻辑完全透明
    Map<String, Object> result = uploadService.uploadAndAnalyze(file);
    return Result.success(result);
}
```

为什么使用 Lua 脚本而不是直接在 Java 代码中操作 Redis?

限流逻辑包含多个 Redis 操作（检查、扣减、回收），如果在 Java 代码中实现，会存在并发问题。

使用 Lua 脚本的优势：

1. 原子性保证

- Lua 脚本在 Redis 中**以单线程模式原子执行**。
- 脚本执行期间，其他命令无法插入。
- 等价于一个事务（但比 Redis 事务更强大）。

2. 减少网络开销

```
Java 多次调用：
  客户端 ---GET---> Redis
  客户端 <--返回--- Redis
  客户端 --ZADD--> Redis
  客户端 <--返回--- Redis
  客户端 --SET---> Redis
  客户端 <--返回--- Redis
  (往返3次，网络延迟 × 3)

Lua 脚本：
  客户端 --脚本+参数--> Redis
         (Redis内部执行GET+ZADD+SET)
  客户端 <-----结果----- Redis
  (往返1次)
```

3. 逻辑集中，易于维护

- 所有限流逻辑在一个 Lua 脚本中。
- 避免 Java 代码和 Redis 操作的混杂。
- 便于版本管理和测试。

性能对比：

方案	网络往返次数	原子性	性能
Java 多次调用	5-10 次	❌	低
Lua 脚本	1 次	✅	高
Redis 事务	2 次	⚠️	中等

关于 Redis 事务的详细介绍，可以参考《Java面试指北》的这篇文章：[Redis常见面试题总结(下)]。

SHA 预加载优化：

为了进一步提升性能，我在启动时预加载脚本：

```java
@PostConstruct
public void init() {
    // 将脚本加载到 Redis，返回 SHA1 哈希值
    this.luaScriptSha =
redissonClient.getScript(StringCodec.INSTANCE).scriptLoad(LUA_SCRIPT);
    log.info("限流 Lua 脚本加载完成，SHA1: {}", luaScriptSha);
}
```

这样做的好处是：

- 减少网络传输（SHA 只有 40 字节，脚本可能有几 KB）。
- Redis 服务器端缓存脚本，执行更快。

请详细解释你的 Lua 脚本的限流算法原理和数据结构设计

算法原理：基于滑动时间窗口的令牌桶算法

我的 Lua 脚本实现了一个**原子化的多维度限流算法**，核心思想是：

1. **预检查阶段**：先检查所有维度是否都有足够的令牌
2. **扣减阶段**：只有所有维度都通过检查后，才统一扣减令牌

这种**两阶段提交**的设计确保了多维度限流的**原子性**。

数据结构设计：

每个限流维度（如 `GLOBAL`、`IP`、`USER`）使用两种 Redis 数据结构：

```
维度 Key: ratelimit:{ClassName:MethodName}:dimension

├── {Key}:value      (String)  ← 实时计数器（当前可用令牌数）
└── {Key}:permits    (ZSet)    ← 时间轴流水账（记录每次令牌分配）
```

1. String (`{Key}:value`) - 实时计数器

- **作用**：快速判断当前是否有足够的令牌
- **初始值**：`max_tokens`（如 5）
- **操作**：
  - 每次请求前检查：`GET {Key}:value`
  - 扣减令牌：`SET {Key}:value (current - permits)`
  - 回收过期令牌：`SET {Key}:value (current + expired_count)`

2. Sorted Set (`{Key}:permits`) - 时间轴流水账

- **作用**：记录每次令牌分配的时间和数量，用于过期令牌回收
- **Score**：请求的时间戳（毫秒）
- **Member**：`request_id:permits`（如 `uuid-123:1`）
  - ⚠ **为什么要加 UUID？** 因为 ZSet 会覆盖相同的 Member。如果不加 UUID，同一毫秒内的多个请求会被合并，导致限流失效。
- **操作**：
  - 记录令牌分配：`ZADD {Key}:permits now_ms "uuid:1"`
  - 查询过期记录：`ZRANGEBYSCORE {Key}:permits 0 (now_ms - interval)`
  - 删除过期记录：`ZREMRANGEBYSCORE {Key}:permits 0 (now_ms - interval)`

为什么使用两阶段提交？

假设有 `GLOBAL` 和 `IP` 两个维度：

- **错误做法**：检查 GLOBAL 通过后立即扣减，再检查 IP（可能失败）
  - 问题：GLOBAL 的令牌已被扣减，但请求最终被拒绝，导致令牌"丢失"
- **正确做法**：先检查所有维度，只有都通过后才统一扣减
  - 好处：保证原子性，要么全部成功，要么全部失败

这种设计思想类似于数据库的两阶段提交（2PC）：

- 阶段1：向所有参与者询问是否可以提交。
- 阶段2：只有所有参与者都同意，才真正提交数据库事务。

Redis Cluster 模式下如何保证限流的正确性？什么是 Hash Tag？

Redis Cluster 将数据分散到 16384 个哈希槽（Slot）中：

- 每个 Key 通过 CRC16 算法计算 Slot：`HASH_SLOT = CRC16(key) % 16384`
- 不同的 Key 可能落在不同的节点上

多维度限流面临的问题：

我的限流方案中，一个方法可能有多个维度（如 `GLOBAL` + `IP`），对应多个 Redis Key：

```
ratelimit:{ResumeController:uploadAndAnalyze}:global:value
ratelimit:{ResumeController:uploadAndAnalyze}:global:permits
ratelimit:{ResumeController:uploadAndAnalyze}:ip:192.168.1.100:value
ratelimit:{ResumeController:uploadAndAnalyze}:ip:192.168.1.100:permits
```

如果这些 Key 落在不同的 Slot（不同的节点），Lua 脚本会报错：

```
(error) CROSSSLOT Keys in request don't hash to the same slot
```

解决方案：Hash Tag 机制

Hash Tag 是 Redis Cluster 提供的一种机制，用于**强制多个 Key 落在同一个 Slot。**

```
Key 格式：prefix{hash_tag}suffix
```

Redis 只对 `{}` 内的内容计算哈希值，`{}` 外的部分被忽略。

为什么设计 count + interval + timeUnit，而不是 permitsPerSecond?

方式1：permitsPerSecond（如 Guava RateLimiter）

```
@RateLimit(permitsPerSecond = 5)
public Result<Void> upload() { ... }
```

特点：

- 简单直观，一个参数搞定。
- 时间单位固定为"秒"。
- 适合大多数场景。

方式2：count + interval + timeUnit（本项目采用）

特点：

- 三个参数组合，灵活性更高。
- 时间单位可自定义（毫秒、秒、分钟、小时、天）。
- 可以表达更复杂的限流规则。

为什么推荐方式 2?

三参数设计更灵活，更贴近业务表达。

如果使用 `permitsPerSecond = 5` 的话，只能表达“每秒 5 次”。而现在我们的设计可以表达：

- 每分钟 100 次
- 每小时 1000 次
- 每 500ms 1 次

☆分布式缓存与会话管理

什么是分布式会话管理？为什么需要？

在单体应用中，用户会话通常存储在应用服务器的内存中（如 `HttpSession`）。但在分布式环境下，用户的请求可能被负载均衡分发到不同的服务器实例，这就带来了**会话不一致**的问题。

传统方案的局限性：

方案	原理	缺点
Sticky Session	负载均衡器将同一用户的请求固定到同一台服务器	服务器宕机会话丢失，负载不均衡
Session 复制	多台服务器之间同步 Session 数据	网络开销大，一致性难保证
集中式存储	将 Session 存储在 Redis 等中间件	增加网络 I/O，但一致性好

本项目采用**集中式存储**方案，使用 Redis 存储面试会话数据，实现跨实例的会话共享。

本项目的会话缓存是如何设计的？

整体架构：

```
用户请求 → 负载均衡 → 任意服务实例 → Redis 读写会话 → 返回响应
                          |
                    ┌─────┴─────┐
                    | Session   |
                    | Cache     |
                    └───────────┘
                          |
                    ┌─────▼─────┐
                    |   Redis   |
                    | (集中存储) |
                    └───────────┘
```

Key 设计规范：

```
# 会话数据（核心）
interview:session:{sessionId}     → JSON(CachedSession)

# 简历-会话映射（支持会话恢复）
interview:resume:{resumeId}       → sessionId
```

设计原则：

1. **业务前缀**：`interview:` 标识业务模块，避免 Key 冲突
2. **层级分隔**：使用 `:` 分隔，便于 Redis 客户端工具按层级展示
3. **唯一标识**：使用 `sessionId` 或 `resumeId` 作为唯一标识

缓存数据结构：

```java
@Data
public class CachedSession {
    private String sessionId;        // 会话唯一标识
    private String resumeText;       // 简历文本（避免重复读取数据库）
    private Long resumeId;           // 关联的简历 ID
    private String questionsJson;    // 序列化的问题列表
    private int currentIndex;        // 当前题目索引
    private SessionStatus status;    // 会话状态（进行中/已完成）
}
```

**为什么把 `questionsJson` ** 存为 String 而不是 List？ **

- Redis String 类型存储 JSON 更简单，避免序列化复杂对象。
- 问题列表在会话开始时生成，后续只读不写，无需频繁修改。
- 减少反序列化开销（只在需要时解析）。

为什么用 Redis 而不是 ConcurrentHashMap?

对比维度	ConcurrentHashMap	Redis
数据共享	仅限单实例	多实例共享
内存占用	占用 JVM 堆内存	独立进程，不影响 GC
持久化	服务重启数据丢失	支持 RDB/AOF 持久化
过期机制	需自己实现	原生 TTL 支持
扩展性	受限于单机内存	可通过集群水平扩展

本项目选择 Redis 的原因：

1. **支持多实例部署**：不同实例可以访问同一份会话数据。
2. **支持会话恢复**：用户可以在不同设备上继续面试（通过 `resume:` 映射）。
3. **自动过期机制**：24 小时后自动清理，无需手动维护。
4. **与其他功能复用**：限流、消息队列都使用 Redis，无需额外引入组件。

> **面试提示**：如果面试官追问"小项目用 Redis 是否过度设计"，可以从**可扩展性**和**组件复用**角度回答。Redis 已经用于限流和消息队列，会话存储只是顺便复用，并没有额外引入复杂度。

缓存过期策略如何设计?

TTL 设置原则：

数据类型	TTL	理由
面试会话	24 小时	用户可能中断面试，次日继续
简历-会话映射	24 小时	与会话生命周期保持一致
限流计数	窗口时间 × 2	确保滑动窗口内的数据完整

过期策略：

Redis 采用**惰性删除 + 定期删除**的组合策略：

- **惰性删除**：访问 Key 时检查是否过期，过期则删除

- **定期删除**：每隔一段时间随机抽取部分 Key 检查并删除过期的

> **注意**：不要依赖 TTL 做业务逻辑判断。如果需要精确控制会话有效期，应在业务层额外校验。

基础缓存操作封装

```
// RedisService.java
public class RedisService {

    // ========== 键值操作 ==========
    public void set(String key, String value, Duration ttl);
    public String get(String key);
    public void delete(String key);
    public boolean exists(String key);
    public void expire(String key, Duration ttl);
    public Long getTimeToLive(String key);

    // ========== Hash 操作 ==========
    public void hSet(String key, String field, String value);
    public String hGet(String key, String field);
    public Map<String, String> hGetAll(String key);

    // ========== 列表操作 ==========
    public void listRightPush(String key, String value);
    public List<String> listGetAll(String key);

    // ========== 原子计数器 ==========
    public Long increment(String key);
    public Long decrement(String key);

    // ========== 分布式锁 ==========
    public boolean tryLock(String lockKey, long waitTime, long leaseTime,
TimeUnit unit);
    public void unlock(String lockKey);
    public <T> T executeWithLock(String lockKey, long waitTime, Supplier<T>
supplier);
}
```

封装的意义：

1. **统一异常处理**：捕获 Redis 连接异常，避免影响主流程。
2. **简化调用**：隐藏 Redisson/Lettuce 的 API 细节。
3. **便于切换**：如果将来更换 Redis 客户端，只需修改这一层。

Redisson 使用

为什么选择 Redisson 而不是 Jedis 或 Lettuce?

特性	Jedis	Lettuce	Redisson
连接模式	同步阻塞	异步非阻塞	异步非阻塞
线程安全	需连接池	线程安全	线程安全
分布式锁	需自己实现	需自己实现	内置 `RLock`
分布式集合	不支持	不支持	`RMap`、`RSet`、`RList`
限流器	不支持	不支持	内置 `RRateLimiter`
Stream 支持	基础 API	基础 API	高级封装
Spring 集成	需手动配置	Spring Data Redis 默认	提供 Starter

本项目选择 Redisson 的原因：

1. **开箱即用的分布式锁**：`RLock` 支持可重入、自动续期、公平锁等特性。
2. **Redis Stream 高级封装**：简化消费者组的创建和消息处理。
3. **Lua 脚本执行**：提供 `RScript` 接口，支持脚本预加载和 SHA 调用。
4. **与 Spring Boot 无缝集成**：通过 `redisson-spring-boot-starter` 自动配置。

> **面试提示**：Jedis 更轻量，适合简单场景；Lettuce 是 Spring Data Redis 的默认实现，性能好但功能基础；Redisson 功能最丰富，适合需要分布式锁、限流等高级特性的场景。

你的项目哪里用到了 Redisson?

1. 分布式限流（Lua 脚本执行）

```
// 预加载 Lua 脚本
@PostConstruct
public void init() {
    this.luaScriptSha = redissonClient.getScript(StringCodec.INSTANCE)
        .scriptLoad(LUA_SCRIPT);
}

// 使用 SHA 执行脚本
Object result = script.evalSha(
    RScript.Mode.READ_WRITE,
    luaScriptSha,
    RScript.ReturnType.VALUE,
    keysList,
    args
);
```

2. Redis Stream 消息队列

```java
// 创建消费者组
RStream<String, String> stream = redissonClient.getStream(streamKey,
StringCodec.INSTANCE);
stream.createGroup(StreamCreateGroupArgs.name(groupName).makeStream());

// 发送消息
stream.add(StreamAddArgs.entries(message).trimNonStrict().maxLen(1000));

// 消费消息
Map<StreamMessageId, Map<String, String>> messages = stream.readGroup(
    groupName, consumerName,
    StreamReadGroupArgs.neverDelivered().count(batchSize)
);
```

3. 分布式锁

```java
// RedisService.java
public boolean tryLock(String lockKey, long waitTime, long leaseTime, TimeUnit
unit) {
    RLock lock = redissonClient.getLock(lockKey);
    try {
        return lock.tryLock(waitTime, leaseTime, unit);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        return false;
    }
}

public void unlock(String lockKey) {
    RLock lock = redissonClient.getLock(lockKey);
    if (lock.isHeldByCurrentThread()) {
        lock.unlock();
    }
}

// 使用示例：防止简历重复分析
public <T> T executeWithLock(String lockKey, long waitTime, Supplier<T>
supplier) {
    if (tryLock(lockKey, waitTime, 30, TimeUnit.SECONDS)) {
        try {
            return supplier.get();
        } finally {
            unlock(lockKey);
        }
    }
    throw new BusinessException("操作正在进行中，请稍后重试");
}
```

Redisson 分布式锁的优势：

- **自动续期（看门狗机制）**：默认 30 秒锁过期，每 10 秒自动续期，防止业务未执行完锁就过期。
- **可重入**：同一线程可多次获取同一把锁。
- **公平锁支持**：`getFairLock()` 按请求顺序获取锁。

详细介绍：

- [分布式锁介绍]

- 分布式锁常见实现方案总结

Redisson 使用中的常见问题

问题 1：锁过期但业务未完成

场景：设置锁过期时间为 10 秒，但业务执行需要 15 秒，导致锁提前释放，其他线程进入。

解决方案：

```
// 方式1：使用看门狗自动续期（不指定 leaseTime）
RLock lock = redissonClient.getLock("myLock");
lock.lock();  // 默认 30 秒，每 10 秒自动续期

// 方式2：合理评估业务耗时，设置足够的 leaseTime
lock.tryLock(10, 60, TimeUnit.SECONDS);  // 等待 10 秒，持有 60 秒
```

问题 2：解锁时抛出 IllegalMonitorStateException

原因：尝试解锁一个不属于当前线程的锁（可能锁已过期被其他线程获取）。

解决方案：

```
// 解锁前检查锁是否属于当前线程
if (lock.isHeldByCurrentThread()) {
    lock.unlock();
}
```

问题 3：Redis 连接池耗尽

现象：高并发时出现 `Unable to acquire connection` 错误。

解决方案：

```
singleServerConfig:
  connectionPoolSize: 128         # 增大连接池
  connectionMinimumIdleSize: 24   # 增大最小空闲连接
  timeout: 10000                  # 增大超时时间（毫秒）
  retryAttempts: 3                # 重试次数
  retryInterval: 1500             # 重试间隔（毫秒）
```

更多内容

缓存常见问题与解决方案、Redis 持久化机制、性能优化等相关问题，可以参考 JavaGuide 的这两篇文章：

- Redis 常见面试题总结（上）（Redis 基础、应用、数据类型、持久化机制、线程模型等）
- Redis 常见面试题总结(下)（Redis 事务、性能优化、生产问题、集群、使用规范等）

另外，《Java面试指北》对 Redis 集群的介绍非常详细，也推荐看看：https://t.zsxq.com/avfM0。